

Addis Ababa University
College of Natural and Computational Sciences
Department of Computer Science

Course Title: **Advanced Database Systems**

Course Number: **CoSc3011**

Credit Hours: 3 ECTS: 5 Contact hrs: 2hrs

Prerequisite: **CoSc2212** (Fundamentals of Database Systems)

Year II

semester II

Course description

Laboratory: 3hrs

Tutorial: 2hrs

This course deals with query optimization, transaction management, recovery and concurrency control, database authorization and security. Additional topics including object oriented, and object relational database systems, distributed databases, and integration may also be covered. A major component of the course is a database implementation project using current database languages and systems.

Course objectives

At the end of this course, students will be able to:

- ✓ Describe the main concepts of the OO model such as object identity, type constructors,
- ✓ encapsulation inheritance, polymorphism, and versioning
- ✓ Evaluate a set of query processing strategies and select the optimal strategy.
- Know the basics
- ✓ of transaction management and concurrency control
- ✓ Understand database security
- ✓ Use different recovery methods when there is a database failure
- ✓ Design a distributed database system in homogenous and heterogeneous environments

Course Outline

Chapter 1: Concepts for Object-Oriented Databases (2 hours)

1.1. Overview of Object-Oriented Concepts

Object-oriented programming is a paradigm centered around objects and classes.

Key concepts include:

- **Objects:** Instances of classes that represent entities with attributes (data) and behaviors (methods).
- **Classes:** Blueprints for creating objects, defining their structure and behavior.
- **Abstraction:** Simplifying complex systems by modeling classes appropriate to the problem.
- **Polymorphism:** Allowing objects to be treated as instances of their parent class, enabling multiple forms.
- **Inheritance:** Mechanism for creating new classes from existing ones, promoting code reuse.
- **Encapsulation:** Bundling data and methods that operate on the data within one unit, and restricting access to some of the object's components.

1.2. Object Identity, Object Structure, and Type Constructors

- **Object Identity:** Each object has a unique identity, distinguishing it from other objects, even if they have the same state.
- **Object Structure:** Refers to the attributes and relationships that define an object's state.
- **Type Constructors:** Mechanisms to define complex data types from simpler ones, such as classes and interfaces.

1.3. Encapsulation of Operations, Methods, and Persistence

- **Encapsulation of Operations:** Hiding the internal implementation details of an object and exposing only the necessary parts through methods.
- **Methods:** Functions defined within a class that describe the behaviors of the objects created from the class.

- **Persistence:** The characteristic of an object to outlive the process that created it, often achieved through serialization or databases.

1.4. Type Hierarchies and Inheritance

- **Type Hierarchies:** Organizing types (classes and interfaces) in a hierarchical structure, where more specific types inherit from more general ones.
- **Inheritance:** A way to form new classes using classes that have already been defined. The new class, known as a subclass, inherits attributes and methods from the existing class, known as a superclass.

Chapter 2: Query processing and Optimization (4 hours)

2.1. Translating SQL Queries into Relational Algebra

Relational algebra is a formal language for the relational model of data. Translating SQL queries into relational algebra involves converting SQL statements into a series of operations on relations (tables). Common operations include:

- **Selection (σ):** Filtering rows based on a condition.
- **Projection (π):** Selecting specific columns.
- **Join ($\bowtie$):** Combining rows from two or more tables based on a related column.
- **Union ($\cup$):** Combining the results of two queries.
- **Difference ($-$):** Finding rows in one query that are not in another.
- **Rename (ρ):** Renaming the output relation.

2.2. Basic Algorithms for Executing Query Operations

Executing query operations efficiently is crucial for database performance. Basic algorithms include:

- **Nested Loop Join:** Iterates over each row of one table and for each row, scans the entire second table.
- **Sort-Merge Join:** Sorts both tables on the join key and then merges them.
- **Hash Join:** Uses a hash table to find matching rows quickly.
- **Index Scan:** Uses an index to quickly locate rows that satisfy a condition.

2.3. Using Heuristic in Query Optimization

Heuristic optimization involves applying rules to transform a query into a more efficient form. Common heuristics include:

- **Push Selections Down:** Apply selection operations as early as possible to reduce the number of rows processed.
- **Combine Selections with Joins:** Merge selection conditions with join conditions to minimize intermediate results.
- **Reorder Joins:** Arrange join operations to minimize the size of intermediate results.

2.4. Using Selectivity and Cost Estimates in Query Optimization

Selectivity and cost estimates help in choosing the most efficient query execution plan:

- **Selectivity:** The fraction of rows that satisfy a condition. Lower selectivity means fewer rows are selected.
- **Cost Estimates:** Predict the resources required (e.g., CPU, I/O) to execute a query. The optimizer uses these estimates to compare different execution plans and choose the least costly one.

2.5. Semantic Query Optimization

Semantic query optimization uses the meaning of the data to optimize queries. It involves:

- **Integrity Constraints:** Using constraints (e.g., primary keys, foreign keys) to simplify queries.
- **Redundant Data Elimination:** Removing unnecessary data or operations that do not affect the result.
- **Query Rewriting:** Transforming a query into an equivalent but more efficient form based on the semantics of the data.

Chapter 3: Transaction Processing Concepts (6 hours)

3.1. Introduction

Transactions are sequences of database operations that are treated as a single unit. They ensure data integrity and consistency, even in the presence of system failures or concurrent access.

3.2. Transaction and System Concepts

- **Transaction:** A logical unit of work that must be either fully completed or fully aborted. It includes operations like read, write, update, and delete.
- **System Concepts:** Involves understanding how transactions interact with the database system, including concurrency control, recovery mechanisms, and transaction management.

3.3. Properties of Transaction

Transactions have four key properties, often referred to as ACID properties:

- **Atomicity:** Ensures that all operations within a transaction are completed successfully. If any operation fails, the entire transaction is rolled back.
- **Consistency:** Ensures that a transaction transforms the database from one consistent state to another.
- **Isolation:** Ensures that the operations of a transaction are isolated from other transactions, preventing concurrent transactions from interfering with each other.
- **Durability:** Ensures that once a transaction is committed, its changes are permanent, even in the case of a system failure.

3.4. Schedules and Recoverability

- **Schedules:** Sequences of operations from multiple transactions. They determine the order in which operations are executed.
- **Recoverability:** Ensures that the database can be restored to a consistent state after a failure. A schedule is recoverable if, for any pair of transactions, if one transaction reads a value written by another, the first transaction commits only after the second transaction commits.

3.5. Serializability of Schedules

- **Serializability:** A schedule is serializable if it results in the same state as some serial execution of the transactions. It ensures that concurrent transactions produce a consistent database state.
- **Types of Serializability:**
 - **Conflict Serializability:** Ensures that conflicting operations (e.g., read/write, write/write) are ordered in a way that maintains consistency.

- **View Serializability:** Ensures that the final state of the database is the same as it would be in some serial execution of the transactions.

3.6. Transaction Support in SQL

SQL provides several features to support transactions:

- **BEGIN TRANSACTION:** Starts a new transaction.
- **COMMIT:** Saves all changes made during the transaction.
- **ROLLBACK:** Undoes all changes made during the transaction.
- **SAVEPOINT:** Sets a point within a transaction to which you can later roll back.

Chapter 4: Concurrency Control Techniques (6 hours)

4.1. Locking Techniques for Concurrency Control

Locking is a fundamental technique to manage concurrent access to data:

- **Shared Locks (S):** Allow multiple transactions to read a data item but not modify it.
- **Exclusive Locks (X):** Allow a transaction to both read and modify a data item, preventing other transactions from accessing it.
- **Two-Phase Locking (2PL):** Ensures serializability by dividing the transaction into two phases:
 - **Growing Phase:** Locks are acquired, and no locks are released.
 - **Shrinking Phase:** Locks are released, and no new locks are acquired.

4.2. Concurrency Control Based on Timestamp Ordering

Timestamp ordering assigns a unique timestamp to each transaction to maintain order:

- **Read and Write Timestamps:** Each data item has read and write timestamps to track the most recent read and write operations.

- **Rules:** Transactions are ordered based on their timestamps to ensure consistency. If a transaction tries to perform an operation that violates the timestamp order, it is aborted and restarted.

4.3. Multi-version Concurrency Control Techniques

Multi-version concurrency control (MVCC) maintains multiple versions of data items to improve concurrency:

- **Versioning:** Each write operation creates a new version of a data item.
- **Read Operations:** Transactions read the most recent version of a data item that existed at the start of the transaction.
- **Write Operations:** Transactions create new versions without affecting other transactions.

4.4. Validation (Optimistic) Concurrency Control Techniques

Optimistic concurrency control assumes conflicts are rare and validates transactions before committing:

- **Phases:**
 - **Read Phase:** Transactions read data items and perform computations.
 - **Validation Phase:** Transactions are validated to ensure no conflicts with other transactions.
 - **Write Phase:** If validation is successful, changes are applied; otherwise, the transaction is aborted and restarted.

4.5. Granularity of Data Items and Multiple Granularity Locking

Granularity refers to the size of the data items being locked:

- **Fine Granularity:** Locking smaller data items (e.g., rows) allows higher concurrency but increases overhead.
- **Coarse Granularity:** Locking larger data items (e.g., tables) reduces overhead but decreases concurrency.

- **Multiple Granularity Locking:** Uses a hierarchy of locks (e.g., database, table, row) to balance concurrency and overhead. Transactions acquire locks at different levels based on their operations.

4.6. Using Locks for Concurrency Control in Indexes

Indexes also require concurrency control to ensure consistency:

- **Index Locks:** Locking index nodes to prevent conflicts during index operations.
- **Key-Value Locking:** Locking specific key-value pairs to allow concurrent access to different parts of the index.
- **Predicate Locking:** Locking ranges of keys to manage range queries and updates.

Chapter 5: Database Recovery Techniques (5 hours)

5.1. Backup and Recovery Concepts

Backup and recovery are critical for ensuring data integrity and availability:

- **Backup:** Creating copies of the database at specific points in time to protect against data loss. Types include full, incremental, and differential backups.
- **Recovery:** Restoring the database to a consistent state after a failure using backups and transaction logs.

5.2. Recovery Concepts Based on Deferred Update

Deferred update (also known as “no-steal” approach) delays the actual updates to the database until a transaction commits:

- **Write-Ahead Logging (WAL):** Ensures that all changes are logged before being applied to the database.
- **Commit Point:** Updates are applied to the database only after the transaction reaches its commit point, ensuring atomicity and durability.

5.3. Recovery Concepts Based on Immediate Update

Immediate update (also known as “steal” approach) applies changes to the database as they occur, even before the transaction commits:

- **Undo and Redo Operations:** If a transaction fails, undo operations revert changes made by the transaction. Redo operations reapply changes from committed transactions to ensure consistency.
- **Checkpointing:** Periodically saving the state of the database to reduce the amount of work needed during recovery.

5.4. Shadow Paging

Shadow paging is an alternative to logging for ensuring database consistency:

- **Shadow Pages:** Maintain two copies of the database pages: the current page and the shadow page.
- **Atomic Updates:** Changes are made to the current pages, and once a transaction commits, the shadow pages are updated atomically to reflect the changes.

5.5. The ARIES Recovery Algorithm

ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) is a comprehensive recovery algorithm:

- **Write-Ahead Logging (WAL):** Ensures that logs are written before changes are applied.
- **Three Phases:**
 - **Analysis:** Identifies the state of transactions at the time of failure.
 - **Redo:** Reapplies all changes from committed transactions.
 - **Undo:** Reverts changes from uncommitted transactions.
- **Checkpointing:** Reduces recovery time by periodically saving the state of the database.

5.6. Recovery in Multi-database Systems

Recovery in multi-database systems involves coordinating recovery across multiple databases:

- **Two-Phase Commit (2PC):** Ensures atomicity across distributed transactions by using a coordinator to manage commit and rollback operations.
- **Global and Local Recovery:** Combines local recovery mechanisms with global coordination to ensure consistency across all databases involved in a transaction.

Chapter 6: Database Security and Authorization (5 hours)

6.1. Introduction to DB Security Issues

Database security involves protecting the database from unauthorized access, misuse, and threats. Key issues include:

- **Confidentiality:** Ensuring that sensitive data is accessible only to authorized users.
- **Integrity:** Protecting data from unauthorized modifications.
- **Availability:** Ensuring that the database is accessible to authorized users when needed.
- **Authentication:** Verifying the identity of users accessing the database.
- **Authorization:** Granting users permission to access specific data and perform certain operations.

6.2. Discretionary Access Control Based on Granting/Revoking of Privileges

Discretionary Access Control (DAC) allows users to control access to their own data:

- **Privileges:** Specific rights to perform operations (e.g., SELECT, INSERT, UPDATE, DELETE) on database objects.
- **Granting Privileges:** Users with sufficient privileges can grant access rights to other users.
- **Revoking Privileges:** Users can revoke previously granted access rights, removing the ability of other users to perform certain operations.

6.3. Mandatory Access Control for Multilevel Security

Mandatory Access Control (MAC) enforces access policies based on security labels:

- **Security Labels:** Classify data and users into different levels (e.g., Top Secret, Secret, Confidential, Unclassified).
- **Access Policies:** Define rules for accessing data based on the security labels of the data and the user.
- **Multilevel Security:** Ensures that users can only access data for which they have the appropriate security clearance, preventing unauthorized access to sensitive information.

6.4. Statistical DB Security

Statistical database security focuses on protecting sensitive information in databases used for statistical analysis:

- **Query Restriction:** Limits the types of queries that can be performed to prevent inference attacks.
- **Noise Addition:** Adds random noise to query results to obscure the exact values of sensitive data.
- **Data Perturbation:** Modifies the data slightly to prevent exact reconstruction of sensitive information while preserving overall statistical properties.

Chapter 7: Distributed Database System (4 hours)

7.1. Distributed Database Concepts

A distributed database is a collection of multiple, interconnected databases spread across different locations. Key concepts include:

- **Distribution Transparency:** Users interact with the distributed database as if it were a single database.
- **Replication:** Storing copies of data on multiple nodes to improve availability and reliability.
- **Fragmentation:** Dividing a database into smaller pieces (fragments) that can be distributed across different locations.
- **Network Transparency:** Hiding the complexities of the network from users and applications.

7.2. Data Fragmentation, Replication, and Allocation Techniques for Distributed Database Design

- **Data Fragmentation:**
 - **Horizontal Fragmentation:** Dividing a table into rows, where each fragment contains a subset of rows.
 - **Vertical Fragmentation:** Dividing a table into columns, where each fragment contains a subset of columns.
 - **Hybrid Fragmentation:** Combining horizontal and vertical fragmentation.
- **Replication:**
 - **Full Replication:** Storing complete copies of the database at multiple sites.

- **Partial Replication:** Storing copies of only some parts of the database at multiple sites.
- **Allocation:**
 - **Centralized Allocation:** Storing the entire database at a single site.
 - **Distributed Allocation:** Distributing fragments of the database across multiple sites based on factors like access patterns and network latency.

7.3. Types of Distributed Database Systems

- **Homogeneous Distributed Databases:** All sites use the same DBMS and have a uniform database schema.
- **Heterogeneous Distributed Databases:** Different sites may use different DBMSs and have different schemas. They require middleware to provide a unified interface.
- **Federated Databases:** A type of heterogeneous database where each site retains control over its own data but cooperates to provide a unified view.

7.4. Query Processing in Distributed Databases

Query processing in distributed databases involves optimizing and executing queries across multiple sites:

- **Query Decomposition:** Breaking down a high-level query into subqueries that can be executed at different sites.
- **Data Localization:** Determining the location of data required to answer the query.
- **Optimization:** Choosing the most efficient way to execute the query, considering factors like data transfer costs and local processing costs.
- **Execution:** Coordinating the execution of subqueries and combining their results to produce the final answer.

Evaluation Schemes

Quiz	10%
Mid Exam	30%
Lab Evaluation and Project Work	20%
Final Exam	40%

Textbooks and References:

Textbook:

Raghu Ramakrishnan, Johannes Gehrke. Database Management Systems, McGraw-Hill; 3rd edition, 2002

Elmasri, R., & Navathe, S. Fundamentals of database systems (7th Edition). Pearson. (2017).

References

1. Osama Mustafa, Robert P. Lockard. (2019). Oracle Database Application Security, Apress, Berkeley, CA.
2. C. J. Date. (2019). Database Design and Relational Theory. 2nd Edition. Apress, Berkeley, CA.
3. Anthony Hack. (2019). SQL Computer Programming for Beginners. Independently published

Laboratory Course Outline

Week 1: Introduction and Software Installation

1.1. (Microsoft SQL Server or Oracle DB)

Introduction to Microsoft SQL Server

Microsoft SQL Server is a relational database management system (RDBMS) developed by Microsoft. It supports a wide range of transaction processing, business intelligence, and analytics applications in corporate IT environments. Key features include:

- **Scalability and Performance:** Efficiently handles large volumes of data and high transaction rates.
- **Security:** Provides robust security features to protect data.
- **Integration:** Seamlessly integrates with other Microsoft products and services.
- **Tools and Services:** Includes tools for data management, development, and business intelligence.

Software Installation

Step 1: Download SQL Server

1. Go to the Microsoft SQL Server download page.
2. Choose the edition you want to install (e.g., Developer, Express).
3. Download the installer.

Step 2: Run the Installer

1. Open the downloaded installer file.
2. Choose the installation type:
 - **Basic:** Quick setup with default settings.
 - **Custom:** Allows you to choose specific features and settings.
 - **Download Media:** Downloads the installation files for offline installation.

Step 3: Follow the Installation Wizard

1. **License Terms:** Accept the license terms and click "Next."
2. **Feature Selection:** Select the features you want to install (e.g., Database Engine Services, SQL Server Replication).

3. **Instance Configuration:** Choose the instance type (default or named instance).
4. **Server Configuration:** Set up the SQL Server services and specify the authentication mode (Windows Authentication or Mixed Mode).
5. **Database Engine Configuration:** Configure the database engine, including data directories and memory settings.
6. **Installation Progress:** Monitor the installation progress and wait for it to complete.

Step 4: Install SQL Server Management Studio (SSMS)

Here's a step-by-step guide to installing SQL Server Management Studio (SSMS):

1. **Go to the SSMS download page:**
 - [Visit the SQL Server Management Studio \(SSMS\) download page¹.](#)
2. **Download and run the SSMS installer:**
 - Click on the download link for the latest version of SSMS.
 - Once the download is complete, locate the installer file (e.g., SSMS-Setup-ENU.exe) in your downloads folder.
 - Double-click the installer file to run it.
3. **Follow the installation wizard to complete the setup:**
 - The installation wizard will guide you through the setup process. Accept the license terms and click "Install".
 - The installer will proceed to install SSMS. This may take a few minutes.
 - Once the installation is complete, click "Close" to exit the wizard.

After installation, you can launch SSMS from the Start menu or by searching for "SQL Server Management Studio".

Step 5: Connect to SQL Server

1. Open SQL Server Management Studio.
 2. In the "Connect to Server" dialog, enter the server name and authentication details.
 3. Click "Connect" to access your SQL Server instance.
1. **Open SQL Server Management Studio:**

- You can find SSMS in the Start menu or by searching for "SQL Server Management Studio".
2. **In the "Connect to Server" dialog, enter the server name and authentication details:**
- **Server name:** Enter the name of your SQL Server instance. This could be localhost for a local instance, or a network name/IP address for a remote server.
 - **Authentication:** Choose the authentication method:
 - **Windows Authentication:** Uses your Windows credentials.
 - **SQL Server Authentication:** Requires a username and password.
3. **Click "Connect" to access your SQL Server instance:**
- Once you've entered the necessary details, click the "Connect" button to establish a connection to your SQL Server instance.

Week 2: Object-Oriented Database

1.1. Object Identity

1.2. Object Query Language (OQL)

Let's explore these object-oriented database concepts:

1.1. Object Identity

Object identity is a fundamental concept in object-oriented databases (OODB). It refers to the unique identifier assigned to each object, which distinguishes it from other objects, regardless of its state or attributes. Key points include:

- **Unique Identifier:** Each object has a unique identifier (OID) that remains constant throughout the object's lifetime.
- **Independence from State:** The identity of an object is independent of its state or attribute values.
- **References:** Objects can reference each other using their unique identifiers, enabling complex relationships and structures.

1.2. Object Query Language (OQL)

Object Query Language (OQL) is a query language designed for object-oriented databases. It allows users to query and manipulate objects in the database. Key features include:

- **Syntax Similar to SQL:** OQL has a syntax that is similar to SQL but is designed to handle complex objects and relationships.
- **Support for Object-Oriented Features:** OQL supports features like inheritance, polymorphism, and encapsulation.
- **Queries on Objects:** Users can perform queries on objects, including selecting, filtering, and joining objects based on their attributes and relationships.

Example of an OQL Query

```
SELECT p.name, p.address  
FROM Person p  
WHERE p.age > 30
```

This query selects the names and addresses of all Person objects where the age attribute is greater than 30.

Week 3: Query Processing and Optimization

3.1. Query Execution Plan

3.2. Query Optimization

Let's delve into query processing and optimization:

3.1. Query Execution Plan

A query execution plan is a sequence of steps used by the database management system (DBMS) to execute a query. It includes:

- **Parsing:** The SQL query is parsed to check for syntax errors and to generate a parse tree.
- **Translation:** The parse tree is translated into a logical query plan, which represents the high-level operations needed to execute the query.
- **Optimization:** The logical query plan is optimized to create a physical query plan, which specifies the most efficient way to execute the query.
- **Execution:** The physical query plan is executed by the DBMS to retrieve the desired results.

Example of a Query Execution Plan

For a query like `SELECT * FROM Employees WHERE Department = 'Sales'`, the execution plan might include:

1. **Scan:** Scan the Employees table.
2. **Filter:** Apply the filter condition `Department = 'Sales'`.
3. **Output:** Return the matching rows.

3.2. Query Optimization

Query optimization is the process of improving the efficiency of a query by choosing the best execution plan. It involves:

- **Cost Estimation:** Estimating the cost of different execution plans based on factors like I/O operations, CPU usage, and memory usage.
- **Heuristics:** Applying rules of thumb to simplify and improve the query (e.g., pushing selections down, combining selections with joins).
- **Transformation Rules:** Using algebraic transformations to rewrite the query into a more efficient form.
- **Plan Selection:** Choosing the execution plan with the lowest estimated cost.

Techniques in Query Optimization

- **Indexing:** Using indexes to speed up data retrieval.
- **Join Optimization:** Choosing the most efficient join method (e.g., nested loop join, hash join, sort-merge join).
- **Subquery Optimization:** Transforming subqueries into joins or other more efficient forms.
- **Parallel Execution:** Splitting the query into parts that can be executed in parallel to reduce execution time.

Week 4: Transaction Management

3.3. Basic Syntax of Transactions

3.3.1. What are Transactions

- 3.3.2. Beginning Transaction
- 3.3.3. Committing or Rolling Back
- 3.3.4. Creating Named Transactions

Week 5: Transaction Management

- 3.4. Making Use of Transactions
 - 3.4.1. Conditionally Committing or Rolling Back
 - 3.4.2. Transactions with Error Handlers

Week 6: Transaction Management

- 3.4.3. Automatic Rollback
- 3.4.4. Nested Transactions

Week 7: Transaction Management

- 3.4.5. Using Save points
- 3.4.6. Transactions in Stored Procedures

Week 8: Concurrency Control Techniques

- 4.1. Locking
- 4.2. Timestamping

Week 9: Concurrency Control Techniques

- 4.3. Multi-version Concurrency Control
- 4.4. Locks for Indexes

Week 10: Database Recovery Techniques

- 5.1. Backup and Recovery Concepts
- 5.2. Backup
- 5.3. Recovery

Week 11: Database Security and Authorization

- 6.1. List the different Types of Accounts in SQL Server or Oracle DB
- 6.2. Explain the use of Server Roles
- 6.3. Demonstrate how to Grant and Deny Permissions
- 6.4. Describe Database Roles
- 6.5. Describe How to Work with Database Permissions

Week 12: Distributed Database

- 7.1. Distributed Database Concepts

- 7.2. Data Fragmentation

- 7.3. Replication

- 7.4. Allocation Techniques

- 7.5. Query Processing in Distributed Databases

Week 13: Triggers

- 8.1. Triggers

Week 14: Stored Procedures

- 8.2. Stored Procedures